CodeQuiz

Assignment 4: Introduction to JavaScript

Astana IT University | Front-End Development / Web Technologies

Objective

Extend the existing four-page CodeQuiz website with interactive JavaScript features: validation, FAQ accordion, popup forms, background colour changes and a live date/time display. Preserve the original quizzes, content, navigation and responsive Bootstrap components.

Team	Members
CodeQuiz	Darkhan Saduakasov; Kassymkhan

Group (complete before submission)

Verified deployed website URL (complete after publishing)

The two blue fields are editable in a PDF reader. The group and public deployment URL were not supplied. The website is packaged locally; it has not been pushed or deployed from this task.

Implementation checklist

Requirement	Implementation / evidence
Task 1 - Form validation	Registration, newsletter and contact checks; screenshot on page 2.
Task 2 - Accordion	Five FAQ questions; smooth opening/closing; page 3.
Task 3 - Popup	Registration and newsletter tabs, three dismissal methods; page 4.
Task 4 - Background colour	Five JavaScript-controlled backgrounds; page 5.
Task 5 - Date and time	Readable Date-based clock refreshed every second; page 5.
Team names / linked scripts	Both names in all four footers; shared local JS/CSS.
Deployment / group	Pending: publish and verify the actual URL; fill the fields above.

Task 1 / Form validation

Implementation: js/script.js - initFormValidation(), getNameError(), getEmailError(), getPasswordError(); shared form markup on all four pages.

Steps

1. Intercept form submission with preventDefault() so invalid input cannot navigate away.
2. Check every required field and write its own error message below the input.
3. Recheck edited fields on input; recheck confirmation whenever the password changes.
4. Focus the first invalid field, or show a local validation success and clear the form.

Input	Rule
Name	At least two letters; Unicode names, spaces, apostrophes and hyphens supported.
Email	Validate local part, domain labels and length; reject dummy/example and known disposable addresses and provider typos.
Password	At least 8 characters, a letter and a number; no spaces, whitespace or invisible characters.
Confirmation / agreement	Exact match to the raw password; required agreement checkbox.

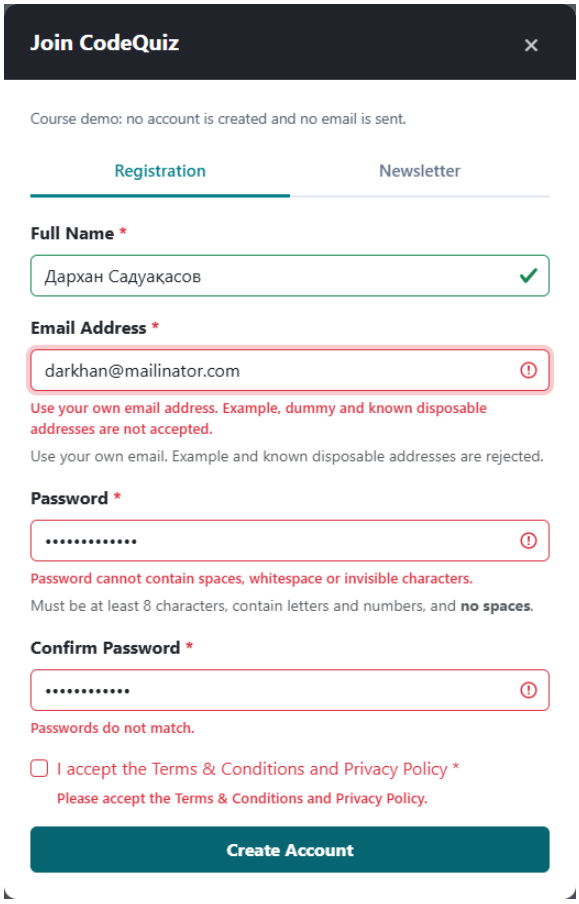


Figure 1. Actual registration error state: disposable email, password whitespace and mismatched confirmation.

Typing, pasting and dropping whitespace are blocked in both password fields; raw submission is checked again. Email format checking cannot prove that a mailbox exists or belongs to a user. Real accounts require a backend verification link/code. No account, email or password storage is simulated as a real service.

Task 2 / Smooth FAQ accordion

Implementation: index.html FAQ section; initFaqAccordion() and .custom-accordion styles.

Steps

1. Keep all five answers collapsed when the page loads.
2. Listen for question button clicks (native buttons also support Enter and Space).
3. Toggle active, aria-expanded and aria-hidden, and set maxHeight from scrollHeight.
4. Close the previous answer when another question opens. Recalculate open height after viewport resizing.

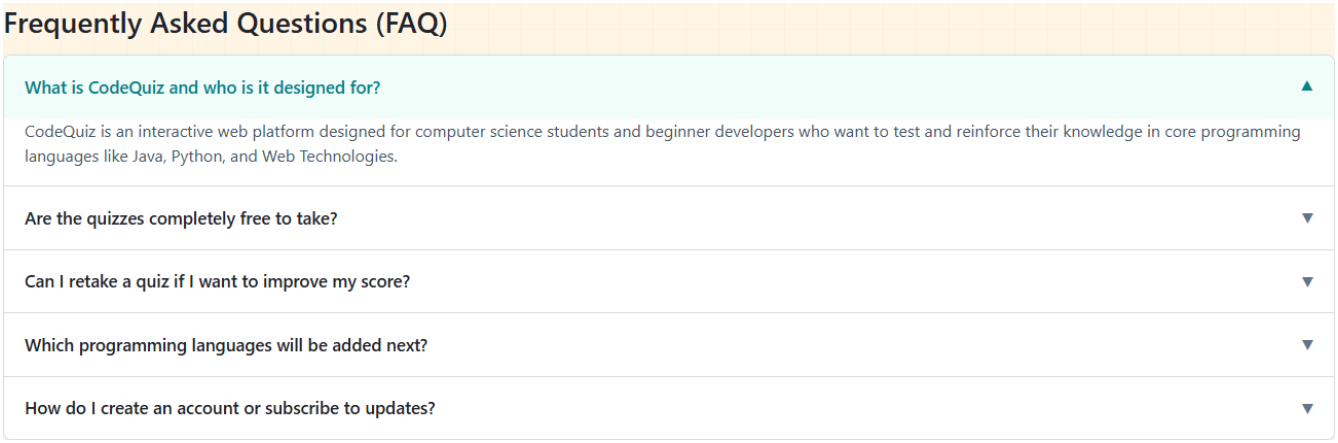


Figure 2. First FAQ answer expanded; the remaining answers stay closed.

Expected behaviour

Click the first question: the answer becomes visible with a CSS height transition. Click it again: it closes. Open another question: only that answer stays expanded. The question button identifies its answer region through aria-controls and a matching ID.

Verification

Browser checks confirmed the default closed state, all five question controls, opening, switching and keyboard closing. The CSS transition remains enabled; reduced-motion preferences shorten animation for users who request it.

Task 3 / Subscription popup

Implementation: shared popup markup on all pages; initPopupModal(); validated newsletter form.

Steps

1. Hide the overlay initially and make its controls inactive.
2. Open the correct tab from Sign Up, Newsletter or Subscribe to Updates.
3. Set dialog state and focus the first input; prevent interaction with the background.
4. Close with the close button, a click on the backdrop or Escape, and restore trigger focus.

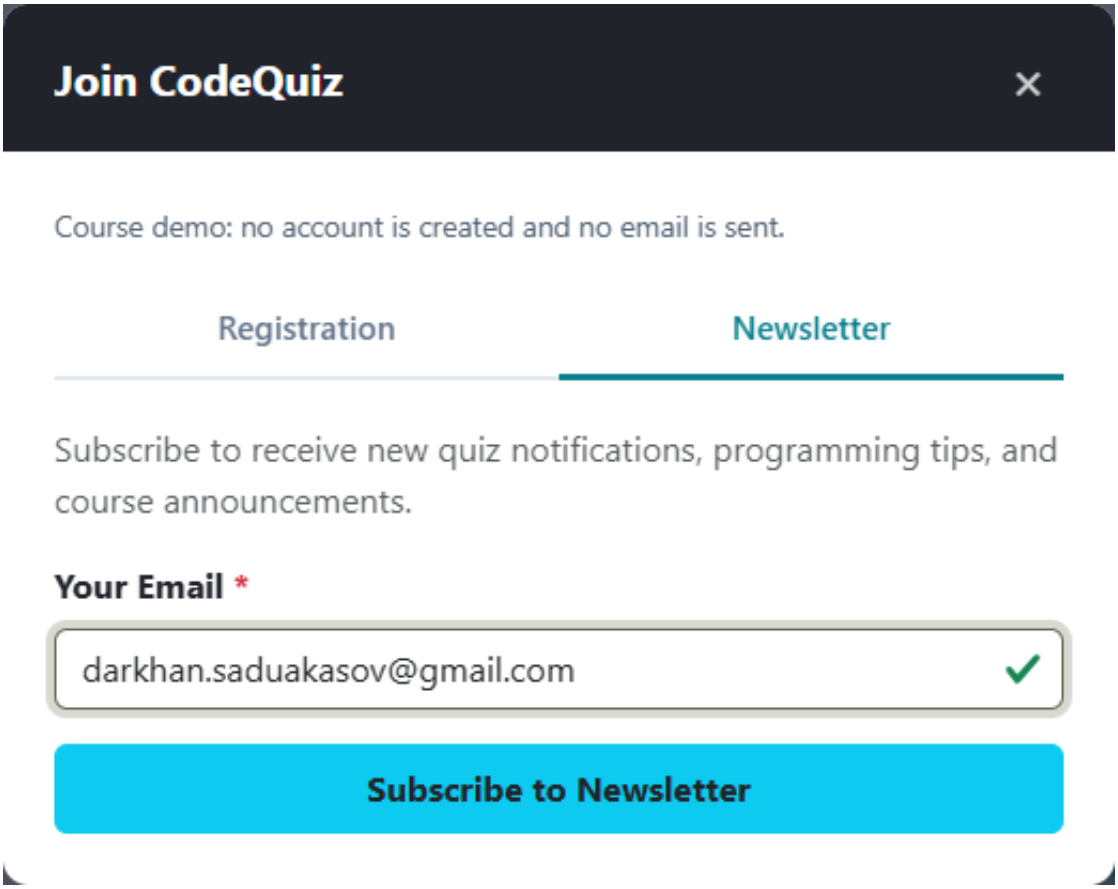


Figure 3. Actual newsletter popup opened from the page. Registration is available in the adjacent tab.

Keyboard and device behaviour

Tab and Shift+Tab stay inside the open dialog. Left/Right/Home/End switch between the two tabs. The form scrolls within the card on smaller screens. Closing restores background access and page scrolling. The desktop and phone layouts were exercised in the automated checks.

Validation remains consistent

Newsletter uses the same email rules as registration and contact, including dummy/disposable address rejection. Its success message states that this is a course demo and does not send newsletters.

Task 4 / Change the background

Implementation: initBackgroundColor(); Change Theme button on all pages.

Steps

1. Define five backgrounds: Code Canvas, Python Mint, Java Amber, Syntax Lavender and Clean White.
2. On click, advance the array index with (index + 1) % palette.length.
3. Assign the selected hex colour to document.body.style.backgroundColor; CSS smooths the change.
4. Save the theme index when storage is available and show a short status notification.

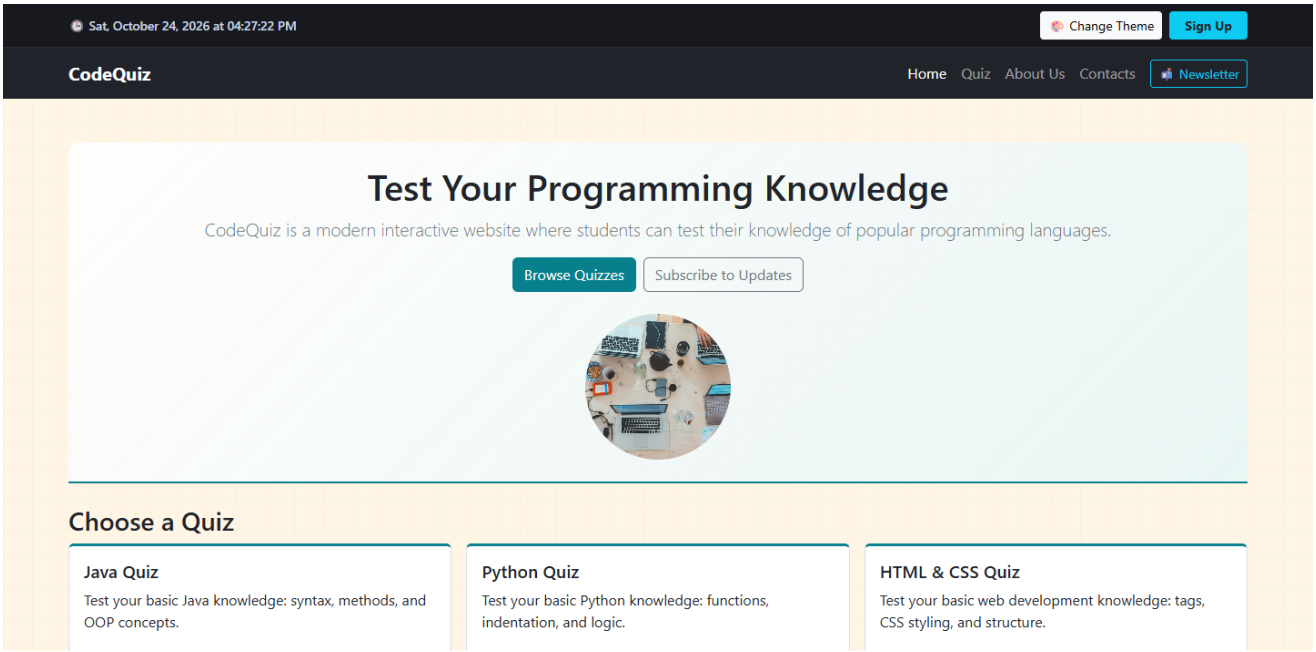


Figure 4. The page after a theme change. The original hero and quiz cards remain.

Task 5 / Current date and time

Implementation: initLiveDateTime(); time#current-date-time on every page.

Steps

1. Create a new Date to obtain the current time.
2. Format it with Intl.DateTimeFormat into readable weekday, date, year and 12-hour time with seconds.
3. Write the visible text and machine-readable datetime value; call the updater every 1,000 ms.

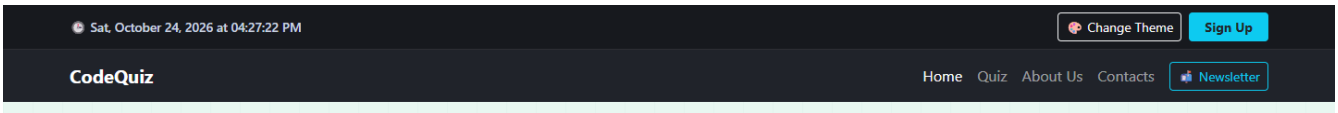


Figure 5. Actual live clock and theme control. The clock uses the viewer's local browser time zone.

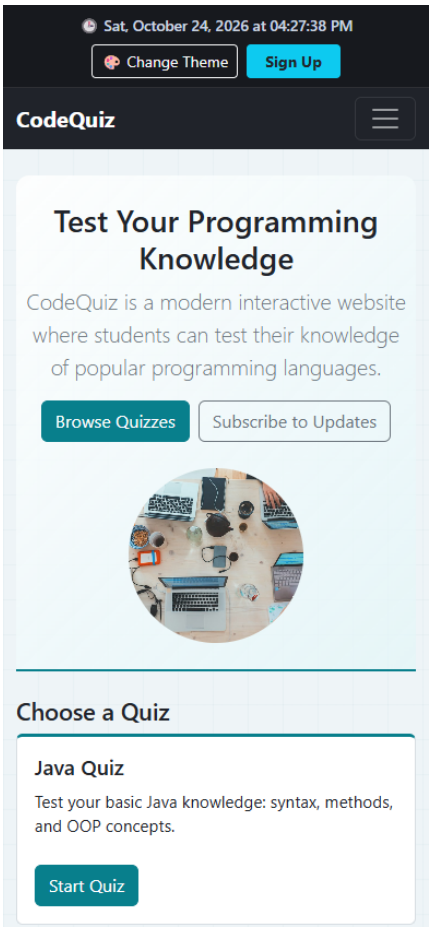
Verification: checks observed the clock change between ticks, different backgrounds, a full five-colour cycle, saved preference and continued operation when localStorage throws an error.

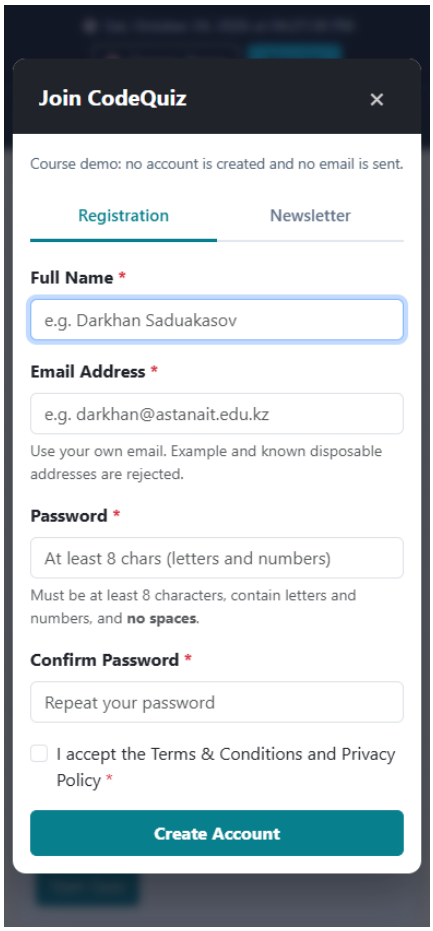
Regression and mobile verification

14 automated test groups passed in Chrome 155.0.8059.85 using offline file URLs. All four website pages were exercised. Test source: tests/browser-tests.cjs; exact results: report/test-results.json.

Preserved functionality

The original Java, Python and HTML/CSS quizzes still reject incomplete answers, grade correct/incorrect choices, display scores and allow resubmission. Navigation, responsive grids, tables, team profiles, topic cards, the image carousel and original PNG assets remain. No input-generated markup is inserted into success messages.





Figures 6-7. Real 390 px viewport screenshots: Home and the scrollable registration popup.

Coverage and limits

Checked widths: 360, 390 and 768 px, plus 1440 px desktop. No horizontal page overflow, missing images, failed asset requests or JavaScript page errors occurred in the completed suite. The mobile About table now scrolls within its own region.

Firefox and Edge could not launch in the restricted Windows test environment; their coverage is not claimed. The runner supports Firefox for an unrestricted machine. Phone screenshots are viewport emulation, not a physical-device test. Recheck the published site on a second browser and real phone before submitting.

Publication, teamwork and reflection

Part 2 / Deployment steps

1. Copy this project into the existing shared repository while keeping its history.
2. Review, commit and push all updated website and report files.
3. Publish the root folder through GitHub Pages or Netlify (see DEPLOYMENT.md).
4. Open the assigned public URL and recheck all four pages and interactive tasks.
5. Enter the verified URL and real group in the first-page fields and editable REPORT.md; save the PDF.

Publication status: pending. No public URL is invented. This report and archive describe the completed local implementation; deployment requires access to the team hosting/repository account.

Existing team responsibility split

Member	Pages and JavaScript components
Darkhan Saduakasov	Home and Quiz; Task 1 validation, Task 5 clock and quiz grading.
Kassymkhan	About Us and Contacts; Task 2 accordion, Task 3 popup and Task 4 background colours.

Preparation for the practice lesson

Each member must understand every component and submit the assignment. Demonstrate empty/invalid and valid forms, blocked password spaces, FAQ opening/closing, all popup dismissal methods, theme changes, the ticking clock and quiz scores. Explain addEventListener, preventDefault, DOM properties, classList, scrollHeight, Date, setInterval and the modulo operation. Kazakh demo notes are included in DEFENSE.md.

Reflection

The update connects HTML structure, CSS presentation and event-driven JavaScript while retaining the previous assignment. Shared validators avoid inconsistent forms, and browser checks revealed real focus, checkbox-layout and mobile-table defects that were corrected. The project also demonstrates the boundary between input validation and real account/email verification: client-side checks improve feedback but do not replace a backend.

Final submission checklist

Fill the group field. Publish and enter the verified public URL. Review the saved report. Submit the report and ZIP before the LMS deadline. Prepare for the defense with both team members.